LIMO Instruction Set Architecture

THE NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science · Faculty of Science and Technology

CS3520 Computer Organization and Architecture

LIMO Group Project · Milestone M1 Submission

Team: Team 7

Lab group: Group C and Group B

Github Repository: https://github.com/realebohamakhaleme68-arch/LIMO-SESOTHO-LANGUAGE-PROCESSOR

#	Member	Student no.	GitHub
1	Realeboha Makhaleme	202322669	realebohamakhaleme68-arch
2	Bahlakoana Mosola	202321284	Kevin-droid408
3	Kobo Ts'epo Ramosoeu	202322608	Kobo-Ramosoeu
4	Katleho Senqhe	202101690	katlehochristophersenqhe-marker
5	Khauhelo Moleboheng Mosanteli	202322649	mosantelimoleboheng-cmd
6	Tumelo Khosola	202321275	khosolatumelo72-alt
7	Mweli Pama	201404452	Mweli Pama

1 Team Charter and Set-up

Project board	https://github.com/users/realebohamakhaleme68-arch/projects/1/views/1
Meeting rhythm	Stand-up Wed/10:10; Thur/20:00; Fri/ 18:00 (15 min: done, next, blockers); sprint review and planning Sunday 15:00. Decisions and action items are logged on the project board.
Decision rule	Discuss until the next stand-up; if still split, a majority vote of all 7 members, with the coordinator breaking ties. The decision and its reason are recorded in the issue.
Definition of done	Merged through a PR with at least one approving review, tests and assembler checks pass, the linked issue is closed, and the spec is updated if the ISA changed.
Branch and PR rules	One issue per branch. Every PR needs one reviewer who is not the author. Direct pushes to main are blocked (branch protection); squash merge.
When a member is absent or not contributing	The member has to give reasons of their absence.

2 ISA Specification

2.1 Overview

	Specification	Rationale
Architecture style	32-bit registers, ALU and data path. Only load(lata) and store(boloka) instructions can directly access memory. ALU instructions use registers only, so each instruction makes at most one memory access.	This keeps the five-stage pipeline simple and gives CPI close to 1.
Register bank	LIMO uses a 32-bit register file containing 16 registers. Each register is 32 bits wide.	The 16-register design provides more general-purpose registers that an 8-register design, reducing register pressure while still

	Specification	Rationale
		keeping the register fields represented using 4 bits, the register identifiers require four bits in the instruction encoding.
Addressing modes	Byte-addressed. Instructions and word accesses are 4-byte aligned. The PC adds 4 each cycle.	Byte addressing matches RISC-V and leaves room for byte loads later
Memory, endianness, alignment	Little-endian: the lowest byte goes at the lowest address. Example: boloka of 0x12345678 at address 0x100 puts 72 at 0x100, 56 at 0x101, 32 at 0x102 and 12 at 0x103. lata and boloka need an address divisible by 4. A misaligned access is undefined in the ISA (LIMO has no exceptions); our simulator reports an error and stops.	Separate instruction and data memories, so IF and MEM can both access memory in the same cycle with no structural hazard
Instruction formats	Fixed 32 bits. R, I, S, B and J (RV32I layouts).	Every instruction is fetched in one cycle and the next PC is PC + 4 unless a branch or jump changes it.
Departures from RV32I	Departure: J is added because qhoma (jal) needs a 21-bit offset, which I-format's 12 bits cannot give. U (lui, auipic) is omitted to keep the set small. Excluded: No multiply/divide, floating point, CSRs, exceptions, flags register or pseudo-instructions	—

2.2 Register File

Number	Sesotho name	Alias	Type	Purpose and convention
r0	noto	x0(zero)	SPR	Hard-wired zero. Reads 0; writes are discarded.
r1	khutla	x1(ra)	SPR	Return address. qhoma writes PC + 4 here. "khutla": return.
r2	lesupa	x2(sp)	SPR	Stack point: address of the top of the stack
r3-r15	lebokose	x3 … x15	GPR	General purpose register. lebokose a place that holds a value
PC	sebali	pc	SPR	Program counter. It is not in the register file and has no register number: it cannot be named as an operand. It is read implicitly by branches, qhoma and qhoma-aterese, and written by them. The address of the instruction being fetched; PC + 4 each cycle

2.3 Instruction Formats

Format	bits 31-25	bits 24-20	bits 19-15	bits 14-12	bits 11-7	bits 6-0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

Immediate ranges: [I: …; S: …; B (branch reach in bytes): …]

2.4 Instruction Set and Sesotho Mnemonics

Class	Mnemonic	Meaning	Syntax	Operation	Fmt	RV32I
Arithmetic	kopanya	to join together, to add	kopanya rd, rs1, rs2	rd ← rs1 + rs2 (wraps on overflow, no flag	R	add

Class	Mnemonic	Meaning	Syntax	Operation	Fmt	RV32I
Arithmetic	tlosa	to take away, remove	tlosa rd, rs1, rs2	rd ← rs1 – rs2	R	sub
Arithmetic	kopanyapalo	kopanya + palo (number): add a number	kopanyapalo rd, rs1, imm	rd ← rs1 + sign-extension(imm12). Also does subtraction by a constant (imm = −1) and register copy (imm = 0)	I	addi
Logic	le	and	le rd, rs1, rs2	rd ← rs1 AND rs2 (bitwise)	R	and
Logic	kapa	or	kapa rd, rs1, rs2	rd ← rs1 OR rs2 (bitwise)	R	or
Logic	kapafeela	kapa + feela (only): “or, but only one”	kapafeela rd, rs1, rs2	rd ← rs1 XOR rs2 (bitwise)	R	xor
Logic	checha-menahane	checha (shift, move) + menahane (to double): shift left, each step doubles	checha-menahane rd, rs1, rs2	rd ← rs1 shifted left by rs2[4:0], zero-filled	R	sll
Logic	checha-arola	checha (shift, move) + arola (to divide): shift right, each step halves a non-negative value	checha-arola rd, rs1, rs2	rd ← rs1 shifted right by rs2[4:0], zero-filled (logical)	R	srl
Logic	papisonyane	papiso (comparison) + nyane (small): is it smaller?	papisonyane rd, rs1, rs2	rd ← 1 if rs1 < rs2 (signed), else 0. Does not jump	R	slt
Memory	lata	to fetch	lata rd, imm(rs1)	rd ← memory word at rs1 + sign-extension(imm12)	I	lw
Memory	boloka	to keep, to store	boloka rs2, imm(rs1)	memory word at rs1 + sign-extension(imm12) ← rs2. Writes no register	S	sw
Branch	tlola-lekana	tlola (jump over) + lekana (be equal)	tlola-lekana rs1, rs2, label	if rs1 = rs2 then PC ← PC + offset, else PC ← PC + 4	B	beq
Branch	tlola-fapana	tlola (jump over) + fapana (differ)	tlola-fapana rs1, rs2, label	if rs1 ≠ rs2 then PC ← PC + offset, else PC ← PC + 4	B	bne
Jump	qhoma	To leap, jump	qhoma rd, label	rd ← PC + 4; PC ← PC + offset (call or jump)	J	jal
Jump	qhoma-aterese	qhoma + aterese (at the address), abbreviated	qhoma-aterese rd, imm(rs1)	rd ← PC + 4; PC ← (rs1 + sign-ext(imm12)) with bit 0 cleared (return or computed jump)	I	jalr

Lexical rules.

- Mnemonics and register names use Lesotho orthography and plain ASCII letters and digits only, so they can be typed on a standard keyboard.
- No mnemonic or register name contains an apostrophe (ts’, ch’) or a special letter such as š. Where the Sesotho word has such a sound, we chose a word without it, or wrote the sound with its plain letters (ts, ch).
- Assembler rule: an apostrophe (’ or ') inside a word is stripped before lookup, so a student who types ts’ gets the same word as ts. This is safe because no two LIMO words differ only by an apostrophe. An apostrophe elsewhere (for example inside a number) is an error with a plain-language message. Matching is case-insensitive.
- Immediates are decimal or 0x hex; labels end with a colon; # starts a comment.

2.5 Encoding Worked Examples

Instruction	Fmt	Field values	Binary (bit 31 … 0)	Hex
kopanya r7, r5, r6 (add x7, x5, x6)	R	funct7 0000000 | rs2 00110 | rs1 00101 | funct3 000 | rd 00111 | opcode 0110011	—	0x00628 3B3
kopanyapalo r5, r0, 10 (addi x5, x0, 10)	I	imm 000000001010 | rs1 00000 | funct3 000 | rd 00101 | opcode 0010011	—	0x00A00 293
lata r8, 0(r9) (lw x8, 0(x9))	I (load)	imm 000000000000 | rs1 01001 | funct3 010 | rd 01000 | opcode 0000011	—	0x0004A 403
boloka r6, 8(r2) (sw x6, 8(x2))	S	imm[11:5] 0000000 | rs2 00110 | rs1 00010 | funct3 010 | imm[4:0] 01000 | opcode 0100011	—	0x00612 423
Tlola-lekana r5, r6, 8 (beq x5, x6, 8)	B	imm[12] 0 | imm[10:5] 000000 | rs2 00110 | rs1 00101 | funct3 000 | imm[4:1] 0100 | imm[11] 0 | opcode 1100011	—	0x00628 463
qhoma r1, 16 (jal x1, 16)	J	imm[20] 0 | imm[10:1] 0000001000 | imm[11] 0 | imm[19:12] 00000000 | rd 00001 | opcode 1101111	—	0x01000 0EF

Opcode and funct assignments

Mnemonic	Format	opcode	funct3	funct7	Rationale
kopanya	R	0110011	000	0000000	RV32I add; same code so Ripes agrees
tlosa	R	0110011	000	0100000	sub differs from add only in funct7[5]
checha-menahane	R	0110011	001	0000000	sll
papisonyane	R	0110011	010	0000000	slt
kapafeela	R	0110011	100	0000000	xor
checha-arola	R	0110011	101	0000000	srl
kapa	R	0110011	110	0000000	or
le	R	0110011	111	0000000	and
kopanyapalo	I	0010011	000	–	Addi: opcode 0010011 is the I-type ALU
lata	I	0000011	010	–	lw: funct3 010 selects a word
qhoma-aterese	I	1100111	000	–	jalr
boloka	S	0100011	010	–	sw: funct3 010 = word
tlola-lekana	B	1100011	000	–	beq
tlola-fapana	B	1100011	001	–	bne: funct3[0] inverts the test
qhoma	J	1101111	–	–	jal

3 Design-Decision Log

(a) Register-file size. What does it cost and buy in encoding bits, forwarding-comparator width and register pressure?

16 registers need 4-bit fields. LIMO still stores 5-bit fields with bit 4 reserved, so no encoding bits are saved today; that is the price of staying bit-identical with RV32I. We gain a half-size register file and 4-bit forwarding comparators (four comparators, 16 XNOR gates instead of 20). Register pressure: 13 general registers remain after noto, khutla and lesupa. Program 1 is our heaviest user, with 11 (r3–r13), so nothing spills. A program with more than 13 live values would spill through lata and boloka.

(b) Architectural or microarchitectural? Which registers are programmer-visible, and which exist only inside the pipeline (IR, IF/ID, …)? Why must the ISA not specify the latter?

Architectural: PC, r0–r15 and memory, the state a program can read or write. Microarchitectural: IR, IF/ID, ID/EX, EX/MEM, MEM/WB, the control signals carried in them, and ForwardA/B. A program cannot name these. Specifying them would forbid other implementations, such as a single-cycle or deeper LIMO, that must run the same binaries. Evidence: our expected results for Programs 1–3 list registers and memory (architectural) separately from cycle counts (pipeline-specific). With forwarding OFF the registers stay the same; only the cycles change.

(c) Branch resolution. In which stage does LIMO resolve branches, and what does that cost in flushed instructions versus extra hardware?

n EX, with always-not-taken prediction. EX reuses the ALU subtraction and its Zero output, so no extra hardware. A taken branch or jump flushes 2 instructions. Program 3 flushes 6 of its 25 cycles: two taken tlolafapana (4) and one qhoma (2). Resolving in ID would flush 1 per taken transfer (3 total) but needs a comparator and forwarding paths in ID. Each tlolafapana directly follows kopanyapalo lb3, so each would stall 1 cycle (3 total): net saving zero here.

(d) Load-use. Why does a load followed by a dependent instruction still stall under full forwarding? Complete the diagram with your own two instructions.

A loaded value exists only at the end of MEM, but the next instruction needs it at the start of EX. Forwarding can carry a value to a later cycle, never back in time, so the consumer must wait one cycle: the hazard unit holds IF and ID and inserts a bubble. In Program 2, kopanya lb7, lb5, lb6 uses lb6 loaded by the lata just before it; Program 2 takes 13 cycles, not 12. Diagram below.

Cycle	1	2	3	4	5	6	7	8
lata lb6, 4(noto)	IF	ID	EX	MEM	WB			
kopanya lb7, lb5, lb6		IF	ID	ID*	EX	MEM	WB	
boloka lb7, 8(noto)			IF	IF*	ID	EX	MEM	WB

* = held for one cycle; a bubble enters EX in cycle 4. In cycle 5, kopanya takes lb6 from the MEM/WB register (ForwardB). Without the stall it would need the value in cycle 4, before the load has produced it

(e) A flags register? RISC-V has none. Would LIMO benefit from one, and what would it do to your hazards?

Not much. RISC-V compares registers inside the branch, so tlolafapana needs no earlier compare. Program 3's loop uses one branch per pass, where a flags design needs a compare first unless the previous ALU instruction sets the flags. Benefits: carry for multi-word arithmetic. Costs: a hidden global register written by many instructions, a new flag data hazard between every ALU instruction and the next branch (more forwarding, one more pipeline register), and flags to save across calls. We would not add one

(f) What breaks first as programs grow: immediate range, branch reach or register count?

Immediate range, because LIMO has no U-format (lui). A 12-bit immediate reaches only ±2047, so constants or data addresses above 2047 need three instructions (kopanyapalo, chechamenahane, kopanya). Our 4 KiB data memory already exceeds that reach from lefela. Register count is next: 13 registers, spilling beyond that. Branch reach (±4 KiB, about 1024 instructions) is last, and tlolalekana can be inverted around a qhoma (±1 MiB) to extend it. Programs 1–3 use offsets under 20 and addresses under 12 bytes, so none of the limits is hit yet.

4 Sample LIMO Programs

Program 1

```
kopanyapalo  lb3, lefela, 12     # lb3 = 12
kopanyapalo  lb4, lefela, 10     # lb4 = 10
kopanya      lb5, lb3, lb4       # 22
tlosa        lb6, lb3, lb4       # 2
le           lb7, lb3, lb4       # 1100 AND 1010 = 1000 = 8
kapa         lb8, lb3, lb4       # 1110 = 14
kapafeela    lb9, lb3, lb4       # 0110 = 6
kopanyapalo  lb11, lefela, 2     # shift amount
chechamenahane lb10, lb3, lb11   # 12 << 2 = 48
chechaarola  lb12, lb3, lb11     # 12 >> 2 = 3
papisonyane  lb13, lb4, lb3      # 10 < 12, so 1
```

Expected final state: lb3 = 12, lb4 = 10, lb5 = 22, lb6 = 2, lb7 = 8, lb8 = 14, lb9 = 6, lb10 = 48, lb11 = 2, lb12 = 3, lb13 = 1. 11 instructions, 0 stalls, 0 flushes, 15 cycles. RAW hazards (lb3/lb4 into kopanya, lb11 into chechamenahane) are all resolved by forwarding.

Program 2: memory and a load-use stall

```
kopanyapalo  lb3, lefela, 5      # lb3 = 5
kopanyapalo  lb4, lefela, 7      # lb4 = 7
boloka       lb3, 0(lefela)      # mem[0] = 5
boloka       lb4, 4(lefela)      # mem[4] = 7
lata         lb5, 0(lefela)      # lb5 = 5
lata         lb6, 4(lefela)      # lb6 = 7
kopanya      lb7, lb5, lb6       # load-use on lb6: 1 stall; lb7 = 12
boloka       lb7, 8(lefela)      # mem[8] = 12
```

Expected final state: lb3 = 5, lb4 = 7, lb5 = 5, lb6 = 7, lb7 = 12; mem[0] = 5, mem[4] = 7, mem[8] = 12. 8 instructions, 1 load-use stall, 0 flushes, 13 cycles.

Program 3: loop, branch taken and not taken, call

```
      kopanyapalo  lb3, lefela, 3          # 0:  counter = 3
      kopanyapalo  lb4, lefela, 0          # 4:  sum = 0
loop: kopanya      lb4, lb4, lb3           # 8:  sum = sum + counter
      kopanyapalo  lb3, lb3, -1            # 12: counter = counter - 1
      tlolafapana  lb3, lefela, loop       # 16: taken twice, then not taken
      tlolalekana  lb4, lefela, ntle       # 20: lb4 = 6, not zero: not taken
      kopanyapalo  lb5, lefela, 99         # 24: lb5 = 99
      qhoma        khutla, ntle            # 28: call; khutla = 32; jumps to 36
      kopanyapalo  lb5, lefela, 1          # 32: skipped (flushed)
ntle: kopanyapalo  lb6, lefela, 7          # 36: lb6 = 7
```

Expected final state: lb3 = 0, lb4 = 6 (3 + 2 + 1), lb5 = 99, lb6 = 7, khutla (r1) = 32. 15 instructions executed; tlolafapana taken 2 times (4 flushes), not taken once; tlolalekana not taken; qhoma taken once (2 flushes); 6 flushes, 0 stalls; 25 cycles (15 + 4 + 6).

References

Computer Organization and Design by David A. Patterson John L. Hennessy.

https://www.geeksforgeeks.org/computer-organization-architecture/microarchitecture-and-instruction-set-architecture/

AI-Use Declaration

ChatGPT, was used as a supplementary tool during the planning, design, and documentation of the LIMO processor project. Its purpose was to support the team's understanding of computer architecture concepts and assist in presenting the project work clearly and systematically.

Sprint Note

Sprint goal: Deliver the M1 ISA specification, with a complete 15-instruction set, hand-encoded examples, the design-decision log and three tested sample programs, and tag it as M1.

Member	Role this sprint	Delivered (issue and PR numbers)	PRs reviewed
Realeboha Makhaleme	Coordinator	Charter, board, M1 tag	none
Bahlakoana Mosola	Overview, registers	Overview, register file	None
Kobo Ts’epo Ramosoeu	Format	Instruction Set and Sesotho Mnemonics	none
Katleho Senqhe	Instruction set	Lexical rules	none
Khauhelo Mosanteli	Encoding	Opcode table	none
Tumelo Khosola	Design log	Diagram and A6	none
Mweli Pama	Programs, references	References, AI-use	none

No commits have been made, our project repository is still local